Gen-AI as a Formal Requirements Engineering Partner: A Case Study in Verified Parser Design

Nicolas Rouquette[1][0000−0003−3137−8690]

Jet Propulsion Laboratory, California Institute of Technology,
Pasadena, CA 91109, USA
nfr@jpl.nasa.gov

Abstract. We report on using generative AI (Gen-AI) as a collaborative partner in formal requirements engineering, demonstrated through three iterative attempts to build a verified YAML 1.2.2 parser in Lean 4. Each architectural failure—monadic Parsec (49% test compliance), parser combinators (context-sensitivity wall), and finally a scanner/token-parser separation (84.9% compliance, 100% error rejection)—revealed formal requirements that only emerged when attempting proofs, not from testing. We identify four Gen-AI roles: (1) *specification analyst*, translating prose into Lean propositions; (2) *architecture critic*, diagnosing structural vs. incidental bugs across failed designs; (3) *proof engineer*, co-developing ∼1,500 theorems about ∼760 definitions; and (4) *pattern analyst*, detecting code patterns that are invisible to testing but break proofs. An ontological classification [10] provides objective verification coverage. A key finding: *proof attempts surface requirements that testing cannot*—five classes of proof-breaking code patterns were identified, all undetectable by any number of tests.

Keywords: Formal requirements · Generative AI · Verified parsing · Lean 4 · YAML · Safety-critical systems

1 Introduction

YAML is a data-serialization language widely used in robotics, DevOps, and mission-critical systems. Its anchor/alias expansion creates Denial-of-Service vectors ("billion laughs" [9]) and language-specific tags enable Arbitrary Code Execution. Using production parsers in safety-critical systems creates supply-chain risk: *unknown vulnerabilities may exist in parsers that pass all tests.*

The YAML 1.2.2 standard [2]—a 100+ page prose document with ∼120 grammar productions—is deliberately implementation-agnostic; extracting formal requirements is a requirements engineering problem, not merely a coding problem. We used Gen-AI (Claude Code with VS Code) as a collaborative partner through three architectural iterations that each revealed requirements invisible to testing.

2 Three Architectures, Three Lessons

The YAML specification is agnostic about parser architecture. Each approach has different implications for verification; Gen-AI helped navigate this design space through three iterations.

Attempt 1: Monadic Parsec (6 days, 49% compliance). Our first attempt [4] used `StateT YamlContext Parser` over Lean's Parsec. It reached 95% on curated examples but 49% on the official test suite [3] (166/337). The problem: Parsec's `attempt` backtracks string position but does *not* restore user-defined state (indentation, flow context, anchors)—rediscovered as a bug source five times, once causing a regression from 230 to 7 passing tests. *Requirements lesson*: `attempt`'s contract excludes user-state restoration, making state-consistency theorems *unprovable by construction*—a missing requirement invisible to testing.

Attempt 2: Parser combinators (lean4-parser). The `lean4-parser` library [6] offered explicit state threading, but hit a wall with *context-sensitive tab processing*. YAML §6.2 permits tabs in some contexts but forbids them in others; in a scannerless parser, character classification depends on state, state depends on previously classified characters, creating a circular dependency that surfaces only when proving classification consistency. Gen-AI proposed the resolution: separate scanning from parsing.

Attempt 3: Scanner + TokenParser (84.9% compliance). The successful design [5] separates a *scanner* (char → token, ∼2,050 lines) from a *token parser* (token → AST, ∼425 lines), with a formal grammar (∼315 lines) as an independent artifact. This enables *layered verification*: character-level predicates, token-level invariants (`Scannable`), and grammar-level predicates (`Grammable`). An *anti-drift mechanism* couples each runtime Bool predicate (e.g., `isWhiteSpaceBool`) with a specification Prop (`isWhiteSpaceProp`) via an iff theorem—if either drifts, the proof fails and the build breaks.

3 Gen-AI as Requirements Engineering Partner

We evolved four complementary Gen-AI roles beyond code generation.

Role 1: Specification Analyst. The AI translated YAML sections into Lean propositions; the human *validated* and *scoped*. A coverage analysis [10] revealed 13 of 19 grammar definitions had zero connecting theorems—the specification existed but was disconnected from the implementation.

Role 2: Architecture Critic. With access to all three codebases, the AI classified bugs as *architectural* vs. *incidental*, identifying Attempt 2's tab-context issue as an architectural barrier, not a proof technique gap.

Role 3: Proof Engineer. Human identifies properties from compiler engineering experience; AI proposes approaches and iterates on type-checker feedback. This produced ∼1,500 theorems about ∼760 definitions.

Role 4: Pattern Analyst. The AI identified five classes of *proof-breaking code patterns*—correct, well-tested code whose structure makes verification intractable, *invisible to testing*:

Table 1. Three attempts. Attempt 2 was abandoned before test-suite evaluation.

Metric	Attempt 1 (Parsec)	Attempt 2 (Combinators)	Attempt 3 (Scan+Parse)
yaml-test-suite	49%	—	84.9%
Error rejection	64%	—	100%
Proved theorems	0	0	∼1,500
Regression tests	62	—	1,041+
Failure mode	State backtrack	Tab context	—

1. **Struct updates before lemmatized calls** prevent lemma unification—lemmas were stated for the original struct.
2. **Weak Head Normal Form (WHNF) expansion targeting inner matches.** Lean's `split` tactic uses WHNF to find the outermost match; internal matches intercept it, silently producing wrong goals.
3. **Branch-product explosion.** N sequential dispatches produce $O(2^N)$ goals; extracting shared sub-computations reduced a ∼320-line proof to ∼30 lines.
4. **Semantic impasse from missing invariants.** A goal $x+1 = x$ signals a missing invariant, not a proof technique gap.
5. **Predicate polarity mismatch.** YAML's Flow-style collections require `Scannable _ true`, but theorems guarantee this only conditionally.

These patterns generalize beyond parsers. Gen-AI both *detected* them and *proposed* mitigations (`generalize` before `split`; decomposition; Wadler-style "free theorems" [8] as refactoring guards).

Verification Coverage Feedback. A *doc-verification-bridge* tool [10] classifies every Lean declaration using Lowe's Four-Category Ontology [11] and automatically infers which definitions each theorem assumes, proves, and validates. Applied to Attempt 3, it classified 1,495 theorems; 264 are *bridging theorems*—capstone proofs connecting Bool computations to Prop specifications—whose inferred assumptions, validations, and dependencies form a natural basis for generating assurance cases or proof blueprints [12].

4 Results and Discussion

Table 1 summarizes the comparison.

Proof as requirements discovery. Each failed proof revealed a requirement (state restoration, context-free character classification, matched brackets, predicate polarity) satisfied by the implementation but never specified—concerning *relationships between definitions*, not I/O behavior. Gen-AI identified refactorings reducing proof complexity from $O(2^N)$ to $O(N)$ [8], but also required correction—occasionally simplifying productions or pursuing brute-force case analysis.

5 Conclusion

Three architectural attempts at a verified YAML parser yielded four contributions: (1) *proof as requirements discovery*—each failure revealed requirements invisible to testing; (2) *five proof-breaking code patterns* generalizing beyond parsers; (3) a *four-role Gen-AI model* structuring collaboration; and (4) *ontological verification coverage* [10,11] providing objective feedback on proven properties. The insight: *formal proof attempts are the most effective requirements elicitation technique for verified systems*, and Gen-AI's greatest leverage is in navigating the design space revealed by proof failures. Future work will exploit the bridging theorems and their dependency graphs for auto-generating assurance cases using proof blueprints [12].

Acknowledgments. This work was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration (80NM0018D0004). Gen-AI assistance was provided through Claude Code in VS Code.

References

1. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Platzer, A., Sutcliffe, G. (eds.) CADE-28. LNCS, vol. 12699, pp. 625–635. Springer (2021)
2. Ben-Kiki, O., Evans, C., döt Net, I.: YAML Ain't Markup Language (YAML) Version 1.2.2. `https://yaml.org/spec/1.2.2/` (2021)
3. yaml-test-suite: Comprehensive test suite for YAML processors. `https://github.com/yaml/yaml-test-suite` (2024)
4. Rouquette, N.F.: lean4-yaml: A YAML 1.2.2 parser for Lean 4. `https://github.com/NicolasRouquette/lean4-yaml` (2026)
5. Rouquette, N.F.: lean4-yaml-verified: A verified YAML 1.2.2 parser in Lean 4. `https://github.com/NicolasRouquette/lean4-yaml-verified` (2026)
6. Dorais, F.G.: lean4-parser: Parser combinator library for Lean 4. `https://github.com/fgdorais/lean4-parser` (2026)
7. Rouquette, N.F.: Make fold combinators total via `Stream.remaining` fuel. Pull request #96, lean4-parser. `https://github.com/fgdorais/lean4-parser/pull/96` (2026)
8. Wadler, P.: Theorems for free! In: FPCA '89, pp. 347–359. ACM (1989)
9. MITRE: CWE-776: Improper restriction of recursive entity references in DTDs ("XML entity expansion"). `https://cwe.mitre.org/data/definitions/776.html`
10. Rouquette, N.F.: What theorems assume, prove, validate, and depend on? `https://github.com/NicolasRouquette/doc-verification-bridge`
11. Lowe, E.J.: The Four-Category Ontology. Oxford University Press (2006)
12. Massot, P.: LeanBlueprint: A tool for formalization-aware mathematical blueprints. `https://github.com/PatrickMassot/leanblueprint` (2024)